Un guide gastronomique

Compétences développées dans le module

- Ecrire des requêtes d'interrogation à une table
  - Projections simples
  - Restrictions
- Ecrire des requêtes en affichant des résultats parlants pour l'utilisateur : renommage des colonnes de la table de résultat, tri des résultats.
- Ecrire des requêtes d'interrogation complexes à plusieurs tables
  - Jointures
- Faire des calculs sur une base de données
  - Avec des fonctions d'agrégation sans partition
  - Avec partition
- Ecrire des requêtes reposant sur des sous-requêtes
  - IN et NOT IN

Le syndicat d'initiative d'une grande ville a décidé de constituer un guide gastronomique des restaurants de la cité.
Ce guide doit renseigner les résidents, habitants et touristes sur les différents plats, et tout particulièrement les spécialités, que l'on peut déguster en ville et sur les vins mis en vente dans les restaurants.

La base de données permet d'avoir :

- Les **restaurants** de la ville, enregistrés avec un code d'identification, un nom, une adresse, les jours d'ouverture, leurs horaires d'ouverture à midi et le soir, la liste des plats proposés à la carte et la liste des vins proposés.
- Les **plats** que l'on peut déguster en ville. Pour chaque plat, on doit connaître son nom, sa place dans le repas (entrée, dessert, ...), la liste des prix dans les restaurants qui le proposent à leur carte, le nom éventuel du vin idéal (d'après un grand gastronome) et une couleur de vin recommandée permettant de choisir un vin adapté dans le cas où l'on ne peut prendre le vin idéal. Cette couleur peut être différente de la couleur du vin idéal.
- Les **vins** que l'on peut trouver en ville avec, pour chacun d'eux, des informations sur leur nom, la date de récolte du raisin, la région d'origine, la couleur, et la liste des prix dans les restaurants qui le proposent.

Voici le schéma relationnel qui a été proposé pour cette base :

Restaurants (code, nom, ville)
Vins (code, cru, millesime, region, couleur)
Plats (code, nom, place, couleurVinRdee, #codeVinIdeal) ServirVin (#codeRestau, #codeVin, prixVin)
ServirPlat (#codeRestau, #codePlat, prixPlat, specialite)
Ouverture (#codeRestau, libJour, horaireOuverture, horaireFermeture)

Ecrire les requêtes SQL qui permettent de répondre aux questions ci-dessous.

TD6 et TP6 : IN & NOT IN

7) Opérateurs de sous-requêtes

Nous avons vu dans le TD précédent le recours aux sous-requêtes pour filtrer des regroupements à partir d'une valeur calculée par sous-requête.

Plusieurs requêtes peuvent être combinées entre elles grâce aux **opérateurs de sous-requêtes** :

- ANY / ALL
- EXIST / NOT EXIST
- IN / NOT IN

Dans cette partie, nous nous intéresserons aux opérateurs **IN** et **NOT IN**.

En SQL, l'opérateur **IN** permet de sélectionner des enregistrements si la valeur de la colonne en question **est comprise** dans **une liste de valeurs**.
Attention : les valeurs doivent appartenir au **même domaine et être du même type de données**.

Si la valeur de la colonne en question correspond à **l'une des valeurs** de la liste, la condition **IN** sera **VRAIE**.

En SQL, l'opérateur **NOT IN** permet de sélectionner des enregistrements si la valeur de la colonne en question **n'est pas comprise** dans **une liste de valeurs**.
Attention : les valeurs doivent appartenir au **même domaine et être du même type de données**.

Si la valeur de la colonne en question ne correspond à **aucune des valeurs** de la liste, la condition **NOT IN** sera **VRAIE**.

***IN et NOT IN* « statique » :**

La liste de valeurs est **définie explicitement** dans la requête.

*Syntaxe **IN** :*

```
SELECT colonne1, …, colonneN
FROM table1
WHERE colonneX  IN ("valeur_1", "valeur_2", …, "valeur_Z")
```

*Syntaxe **NOT IN** :*

```
SELECT colonne1, …, colonneN
FROM table1
WHERE colonneX  NOT IN ("valeur_1", "valeur_2", …, "valeur_Z")
```

***IN et NOT IN* « dynamique » :**

La liste de valeur provient d'une **liste de valeurs renvoyée par une sous-requête**

*Syntaxe **IN** :*

```
SELECT colonne1, …, colonneN
FROM   table1
WHERE  colonneX  IN ( SELECT colonneY
                      FROM table2
                      [WHERE….])
```

*Syntaxe **NOT IN** :*

```
SELECT colonne1, …, colonneN
FROM   table1
WHERE  colonneX  NOT IN ( SELECT colonneY
                          FROM Table2
                          [WHERE….])
```

Attention :

- colonneY doit être du même type que colonneX.
- colonneY peut être :
  - soit une colonne normale,
  - soit le résultat d'une fonction d'agrégation (comme MAX(), AVG(), etc.).

IN et NOT IN statique

Q33. Les restaurants du BAB: "Bayonne", "Anglet" ou "Biarritz" ?
Q34. Cru, couleur et millésime de tous les vins, sauf ceux issus des régions "TRAN", "GERS" et "ALSACE" ?
Q35. Les plats servis comme plat de résistance ou entrée ?
Q36. Les codes des restaurants qui ouvrent le week-end (samedi ou dimanche) ?

IN et NOT IN Dynamique

Q37. Les crus des vins d'ALSACE ayant le même millésime que le plus ancien des vins de BORDEAUX ?
Q38. Noms des restaurants qui proposent un "Plateau de fruits de mer", mais qui ne proposent pas le meilleur vin recommandé avec ce plat ?
Q39. Cru, millésime et couleur des vins qui ne sont recommandés avec aucun plat ?
Q40. Noms et adresses des restaurants qui proposent à la fois "Pizza" et "Pâtes fraîches" ?

Les sous-requêtes peuvent aussi être utilisées dans une clause HAVING.

En SQL, les sous-requêtes ne se limitent pas à la clause WHERE.
Elles peuvent également être utilisées dans la clause HAVING, notamment lorsqu'on souhaite filtrer des groupes en comparant une fonction d'agrégation avec une valeur calculée dynamiquement.

*Syntaxe **IN** :*

```
SELECT
FROM
[WHERE…]
GROUP BY
Having FONCTION(colonne) IN ( SELECT colonneY FROM table2 [WHERE….])
```

*Syntaxe **NOT IN** :*

```
SELECT
FROM
[WHERE…]
GROUP BY
Having FONCTION(colonne) IN ( SELECT colonneY FROM table2 [WHERE….])
```

Q41. Noms des restaurants qui proposent plus de plats que le restaurant "Zokoak".